Final Submission

FlyRank Internship - Backend Track - Week 1 Assignment A3

Containerize your stack

Generated: 2026-09-06

Project

FlyRank Task API - Containerized Postgres. This submission uses Python, FastAPI, psycopg 3, PostgreSQL 16, Docker, and Docker Compose.

Repository State

- Clean worktree, excluding generated submission PDF assets
- Git remote: No remote configured in this local clone. Add the public GitHub URL before submitting if required by the portal.

Important Note

- A genuine database screenshot is not present at docs/postgres-data.png. Docker is also unavailable in this environment, so this PDF does not fabricate psql output.

Requirement Evidence

- Dockerfile present: Dockerfile builds the FastAPI app image and starts uvicorn on port 8000.
- Compose stack: compose.yaml defines api and db services.
- Postgres container: db uses the official postgres:16 image.
- Persistence: compose.yaml defines and mounts the named taskdata volume at /var/lib/postgresql/data.
- Environment config: .env is git-ignored and .env.example documents POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB, and DATABASE_URL.
- Parameterized queries: repository.py uses psycopg %s placeholders for SELECT, INSERT, UPDATE, and DELETE inputs.
- Startup database setup: repository.init_db creates the tasks table if missing and seeds three tasks only when the table is empty.
- CRUD endpoints: main.py exposes GET /tasks, GET /tasks/{id}, POST /tasks, PUT /tasks/{id}, and DELETE /tasks/{id}.
- Status codes: Successful routes return 200, 201, or 204; invalid input returns 400; missing tasks return 404.
- Documentation: README.md includes one-command run instructions, endpoint table, curl example, and database verification notes.
- Commits: git log contains six A3 stage commits.

Latest A3 Commits

```
e849143 Stage 5: one-command stack + docs
7bc519d Stage 4: docker-compose the whole stack
7586ce5 Stage 3: full CRUD on Postgres
b462ded Stage 2: read from Postgres
c2616b9 Stage 1: connect via env and create table
6a8ac0e Stage 0: Postgres in Docker + gitignore
```

Run Instructions

A reviewer can run the project from a clean clone with these commands:

```
Copy-Item .env.example .env
docker compose up --build
```

API URL: http://localhost:8000

Swagger UI: http://localhost:8000/docs

Database Verification

```
docker exec -it taskdb psql -U postgres -d tasks
\dt
SELECT * FROM tasks;
```

Persistence Check

Create a task, run docker compose down, then run docker compose up --build again. The named taskdata volume should keep the row available through GET /tasks.

Key Files

compose.yaml

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL:
postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16
    container_name: taskdb
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - taskdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 2s
      timeout: 5s
      retries: 15

volumes:
  taskdata:
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

.env.example

```
POSTGRES_USER=postgres
POSTGRES_PASSWORD=dev
POSTGRES_DB=tasks
DATABASE_URL=postgresql://postgres:dev@localhost:5432/tasks
```

API And Database Evidence

Endpoints From README

GET /, GET /health, GET /tasks, GET /tasks/{task_id}, POST /tasks, PUT /tasks/{task_id}, DELETE /tasks/{task_id}.

curl-output.txt

```
$ curl -i http://localhost:8000/tasks
HTTP/1.1 200 OK
content-type: application/json

[{"title":"Learn FastAPI","done":false,"id":1},{"title":"Build CRUD
endpoints","done":false,"id":2},{"title":"Test with Swagger","done":false,"id":3}]
```

crud-checklist.txt

```
Stage 3 checkpoint

1. POST /tasks with {"title":"Docker task"} -> 201
2. GET /tasks -> task is present
3. PUT /tasks/{id} with {"title":"Docker task done","done":true} -> 200
4. GET /tasks/{id} -> updated task
5. DELETE /tasks/{id} -> 204
6. GET /tasks/{id} -> 404
```

postgres-read-check.txt

```
Stage 2 checkpoint

curl -i http://localhost:8000/tasks
curl -i http://localhost:8000/tasks/999

Expected: 200 with rows from Postgres, then 404 with a task-not-found JSON error.
The repository uses parameterized queries with %s placeholders.
```